Iroi

- A Multimodal Robot Assistant forHands-on Lab Learning

March 2026
Version No.01.03

Xinyi Hu	MSTI Robotics
Linzhengrong Shao	MSTI Robotics
Yifan Li	MSTI Robotics
Luyao Niu	Instructor
Mark Licata	Instructor

DOGU

UNIVERSITY of WASHINGTON | GLOBAL INNOVATION EXCHANGE

1. Product Overview

Iroi is an AI-powered robot assistant for hands-on laboratory courses. It patrols the lab, detects raised hands, navigates to students, and provides voice-based, image-aware guidance grounded in official course materials.

1.1 The Problem

In lab sessions, students often need immediate help. Instructors and TAs cannot be everywhere at once, and general-purpose AI tools lack course context and accept only text. Students are left waiting or guessing.

1.2 What Iroi Does

Iroi autonomously patrols the lab, detects raised hands, and navigates to the student. It accepts spoken questions and photos of student work, then provides step-by-step guidance strictly based on official course materials. It guides reasoning rather than giving direct answers.

1.3 Who It Is For

- Students in hands-on lab courses needing real-time help
- Instructors and TAs who want to support more students at once
- Academic institutions integrating AI into physical lab environments

1.4 Primary Features

Feature	Description
On-Demand Assistance	Detects raised hands and navigates to the student automatically.
Multimodal Interaction	Accepts spoken questions and photos of circuit diagrams or setups.
Instructional Guidance	Walks students through reasoning step by step, not direct answers.
Grounded Knowledge Base	Responses strictly based on official lab Markdown files.
Wake-Word Activation	Say "Hi Iroi" to start a conversation.
Robot Navigation	Uses 3D LiDAR, 2D LiDAR, and ultrasonic sensors to navigate safely.

2. System Requirements

2.1 Package Contents

- 1x DOGU Iroi robot unit (pre-configured)
- 1x External USB webcam (Logitech Brio or equivalent)
- 1x Power cable and charging dock

- 1x Wi-Fi router or access point (if not provided by institution)
- 1x Quick-start guide (printed)
- DOGU Planner access credentials (provided via email)

2.2 Hardware Requirements

Component	Specification
Robot model	DOGU Iroi
Onboard computer	Integrated Linux-based computing unit
Sensors	3D LiDAR, 2D LiDAR, ultrasonic sensor, front-facing camera
Audio I/O	Built-in microphone and speaker
External camera	USB webcam connected to onboard computer
Display	Integrated touchscreen
Environment	Indoor flat surfaces; minimum 1.5 m aisle width

Optional deployment computer (for map creation): Windows 10/11 or Ubuntu 20.04+, 4 GB RAM minimum, 8 GB recommended, Wi-Fi or Ethernet on the same network as the robot.

2.3 Software Requirements

Operating system (onboard): Ubuntu 20.04 LTS with ROS (pre-installed).

Package	Install
Python 3.10+	Pre-installed
openai	pip install openai
openwakeword	pip install openwakeword
PyQt6	pip install PyQt6
numpy	pip install numpy
mpg123	sudo apt install mpg123
ALSA / arecord	Pre-installed on Ubuntu

2.4 Internet and Accounts

An active internet connection is required at all times. All AI features depend on OpenAI cloud APIs. There is no offline fallback. Minimum recommended: 10 Mbps stable connection.

- OpenAI account with API key — access to GPT-4o, Whisper, and TTS endpoints
- DOGU account — provided by the industrial sponsor; required for Planner and robot configuration

3. Installation and Setup

3.1 Physical Setup

1. Unbox Iroi and verify all package contents.
2. Place the charging dock near an AC outlet and plug it in.
3. Connect the USB webcam to the robot's onboard computer.
4. Power on the robot. Wait ~60 seconds for the home screen to appear.

3.2 Network Configuration

1. Connect the robot to the lab Wi-Fi via the touchscreen or SSH.
2. Connect the administrator computer to the same network.
3. Verify connectivity by pinging the robot's IP address.

The robot's IP address is found in the touchscreen system settings.

3.3 Software Installation

All steps below run on the robot's onboard computer (via SSH or direct terminal).

Step 1 — Clone the repository:

```
git clone <github-repo-url>
```

Step 2 — Install Python dependencies:

```
pip install -r requirements.txt
```

Step 3 — Set the OpenAI API key:

```
echo 'export OPENAI_API_KEY="your-api-key-here"'
```

The API key must be set as an environment variable. Never hardcode it in source files or commit it to version control.

Step 4 — Add knowledge base files:

Copy all course Markdown (.md) files to docs/. All .md files under any subdirectory are loaded automatically at startup.

3.4 Map Creation with DOGU Planner

1. Install DOGU Planner on the administrator computer.
2. Launch Planner and connect to the robot using its IP address.

3. Use manual drive mode to drive the robot around the full lab perimeter. LiDAR builds the map automatically.
4. Save the map and define waypoints (e.g., Lab Station 1, Charging Dock).
5. Upload the map to the robot via Planner.

Waypoint names in Planner must match those in the command_parser module's WAYPOINT_MAP.

4. Operating Instructions

4.1 Starting the System

1. Power on the robot and wait for the home screen (~60 seconds).
2. Confirm the robot is connected to the lab Wi-Fi.
3. Check the battery level (recommended: above 50%).
4. Launch the assistant:

```
python3 robot_ui.py --voice-mode realtime_tts
```

The PyQt6 GUI will appear and the system begins listening for the wake word.

4.2 Activating the Robot

Hand raise: Raise your hand clearly in view of the robot's camera. Iroi detects it via YOLO perception and navigates to you. The screen will show HOW CAN I HELP YOU.

Wake word: Say "Hi Iroi" when the robot is nearby. The robot acknowledges and displays the home screen.

4.3 Using the Q&A Interface

1. Say "Hi Iroi" or tap the screen to wake the assistant.
2. Select the relevant lab section on the Menu Screen (e.g., LAB 1, LAB 2).
3. Ask a question by voice or submit an image of your setup.

Voice input: Speak your question naturally. Iroi transcribes it, displays it on screen, and responds with both text and audio.

Image input: Tap the image upload button, point the webcam at your setup, tap Confirm, and ask your question. Iroi analyzes the image against course materials. Supported formats: JPEG, PNG, WebP, GIF.

4.4 Navigation Commands

Instructors and admins can issue robot navigation commands by voice:

Command	Action

"Go to [waypoint name]"	Navigate to the specified location
"Come to me"	Robot navigates toward the speaker
"Stop"	Robot halts all movement immediately

4.5 Ending a Session

1. Press Ctrl+C in the terminal or close the GUI to stop the assistant.
2. Return the robot to the charging dock. Confirm the charging indicator is active.
3. Power off the robot if it will not be used again that day.

All conversation history and session state are discarded on exit. No data is stored between sessions.

5. Technical Specifications

5.1 System Architecture

Layer	Description
Application	VoiceAssistantRAG class orchestrates the full pipeline: wake word → STT → RAG → TTS → playback.
Voice Interaction	Audio captured via arecord (ALSA), wake detection via openWakeWord, transcription via Whisper, playback via mpg123.
AI / RAG	GPT-4o with full Markdown knowledge base injected into every system prompt. Supports text and image input.
Knowledge Base	All .md files under TechRAG/docs/ loaded at startup and cached as a single string.
Robot / ROS Control	Commands parsed by command_parser into intents (navigate, move_to_user, stop) and sent to ros_command.
UI / Signal	PyQt6 signals: wake_word_detected, speech_recognized, response_ready, speaking_finished.

5.2 Technologies Used

Category	Technology
Language	Python 3
LLM	GPT-4o (OpenAI Python SDK)
Wake Word	openWakeWord — phonetic trigger for "iroi"
STT	OpenAI Whisper API
TTS	OpenAI TTS API
GUI	PyQt6

Audio Recording	ALSA / arecord (Linux only)
Audio Playback	mpg123 (preferred) or ffplay (fallback)
Robot Control	ros_command (custom module)
Command Parsing	command_parser (custom module)

5.3 Key Configuration Parameters

Parameter	Default	Description
chat_model	gpt-4o	LLM model identifier
user_record_seconds	12	Max recording duration per turn
oww_threshold	0.1	Wake word confidence threshold
energy_threshold	300	Minimum audio energy to process
tts_voice	alloy	TTS voice profile
docs_dir	TechRAG/docs/	Knowledge base root path

5.4 Data Storage

No database is used. The knowledge base consists entirely of Markdown files on the local filesystem. At startup, all files are loaded into memory as a single concatenated string (self.docs_context) and injected into every API call. All runtime state is in-memory only and not persisted between sessions.

Recommended future upgrades: SQLite or PostgreSQL for conversation logging; a vector database (Chroma, FAISS) for semantic retrieval instead of full-corpus injection.

6. Safety and Privacy

6.1 Data Privacy

- Audio: Temporary files are deleted immediately after use. Audio is sent to OpenAI Whisper and is subject to OpenAI's privacy policy.
- Images: Encoded as base64 and sent to GPT-4o. Not stored permanently. Avoid submitting sensitive personal data.
- Knowledge base: Full content is sent to OpenAI with each API call. Do not place exam answers, personal student data, or proprietary content in TechRAG/docs/.
- Conversation logs: Not persisted. All session data is discarded on exit.

6.2 Security Warnings

- API key: Load from environment variable only. Never hardcode in source files or commit to version control. Exposure can result in unauthorized usage and financial liability.
- Robot command injection: Anyone within microphone range can issue commands after wake-word activation. Commands are limited to pre-approved intents by command_parser.
- Wake word vulnerability: No speaker verification. Any sufficiently similar sound can activate the assistant.
- Cloud dependency: All AI functionality fails if the OpenAI API is unavailable. No offline fallback exists.

6.3 Known Operational Limitations

- No hardware emergency stop. If the voice pipeline fails mid-operation, robot movement cannot be halted by voice.
- Self-interference mitigation: The is_speaking flag delays recording during TTS playback, but echo in reflective environments may still cause false activations.
- No knowledge base sanitization: Malformed or adversarially crafted Markdown files could alter LLM behavior.

7. Troubleshooting

7.1 Audio and Microphone

arecord fails:

- Run arecord -l to list capture devices and verify the device name.
- Check user audio permissions: sudo usermod -aG audio $USER (requires re-login).
- On macOS, arecord is not available. Use sounddevice, sox, or ffmpeg instead.

Wake word never triggers:

- Verify oww_model loaded successfully at startup (check logs).
- Confirm microphone audio energy exceeds 300 (test with _check_audio_energy()).
- Ensure the openWakeWord model correctly recognizes the phonetic trigger for "iroi".

7.2 LLM and RAG

Assistant fails to connect:

- Check the OPENAI_API_KEY environment variable is set and valid.
- Verify network access to api.openai.com.

Incorrect or knowledge-base-ignoring responses:

- Confirm .md files exist in TechRAG/docs/ and contain expected content.
- Check startup logs for any Warning: Failed to load ... messages.

- If the knowledge base is very large, it may exceed the model's context window. Consider splitting documents.

7.3 Robot and ROS

Robot does not move after a command:

- Check COMMAND_PARSER_AVAILABLE is True.
- Confirm commander.initialized is True.
- Verify the waypoint name exists in the robot's navigation map.

Command parser classifies everything as a question:

- Test parse_command() in isolation with known phrases.
- Review regex patterns in command_parser.COMMAND_PATTERNS and WAYPOINT_MAP. Phrasing must match exactly.

7.4 GUI and Startup

App does not exit after session:

- Confirm set_app_instance(app) was called before start_listening().

ImportError for command_parser or ros_command:

- This is expected and handled gracefully. COMMAND_PARSER_AVAILABLE is set to False. Q&A functions normally; robot control is unavailable.

8. Known Limitations

8.1 Knowledge Base

- Full-corpus injection: All Markdown files are injected into every prompt. As the knowledge base grows, this increases API cost and may exceed GPT-4o's ~128K token context window.
- No live updates: Knowledge base is loaded once at startup. Restart required to reflect file changes.
- No conversation memory: Each query is stateless. Follow-up questions require full re-statement of context.

8.2 Voice Pipeline

- Linux only: Full voice mode requires ALSA / arecord. macOS and Windows are not supported without replacing the audio subsystem.
- Fixed recording duration: User speech is capped at 12 seconds. No voice activity detection (VAD).

- One-shot model: After one wake → question → answer cycle, the app exits. No continuous multi-turn conversation without relaunch.
- No offline fallback: All AI features (STT, LLM, TTS) require internet access.

8.3 Robot Control

- Limited command set: Only navigate, move_to_user, and stop are supported.
- No status feedback: The system sends commands but does not receive or relay robot status updates.
- Command parser reliability: Unrecognized commands silently fall through to Q&A with no indication of failure.

8.4 Planned Features

Feature	Status	Note
Voice Activity Detection (VAD)	In Progress	Reducing fixed 12s recording window; adaptive end-of-speech detection under development.
Speaker Verification	Not Started	Planned to restrict activation to registered users only.
Teacher Dashboard	Not Started	Web interface for instructors to monitor session activity and student interactions.
Student Profile	Not Started	Per-student interaction history and progress tracking across lab sessions.
Persistent Conversation Logging	Not Started	Session logs to be stored in SQLite or PostgreSQL for analytics.